

AI Assisted Coding Lab 11.4

Name: P.Ayushman

2303A52294

Batch:42

Task 1: Stack Implementation for Undo Operations (LIFO)

Scenario:

You are building a text editor where users can undo their recent actions (typing, deleting, formatting). Each action must be reversed in Last-In-First-Out (LIFO) order

Prompt: Stack Implementation for Undo Operations (LIFO)

```
1  #Stack Implementation for Undo Operations (LIFO)
2  class Stack:
3      def __init__(self):
4          self.stack = []
5      def push(self, item):
6          self.stack.append(item)
7      def pop(self):
8          if not self.is_empty():
9              return self.stack.pop()
10         else:
11             raise IndexError("Stack is empty")
12     def peek(self):
13         if not self.is_empty():
14             return self.stack[-1]
15         else:
16             raise IndexError("Stack is empty")
17     def is_empty(self):
18         return len(self.stack) == 0
19     def size(self):
20         return len(self.stack)
21 # Example usage of Stack for undo operations
22 if __name__ == "__main__":
23     undo_stack = Stack()
24     # Simulating some operations
25     undo_stack.push("Operation 1")
26     undo_stack.push("Operation 2")
27     undo_stack.push("Operation 3")
28     print("Current Stack:", undo_stack.stack)
29
30     return len(self.stack)
31 # Example usage of Stack for undo operations
32 if __name__ == "__main__":
33     undo_stack = Stack()
34     # Simulating some operations
35     undo_stack.push("Operation 1")
36     undo_stack.push("Operation 2")
37     undo_stack.push("Operation 3")
38     print("Current Stack:", undo_stack.stack)
39     # Undo last operation
40     last_operation = undo_stack.pop()
41     print("Undoing:", last_operation)
42     print("Current Stack after undo:", undo_stack.stack)
```

OUTPUT:

Scenario

You are designing a music playlist feature where songs can be added or removed dynamically while maintaining order.

Prompt: Singly Linked List for Dynamic Playlist Management

```
65 #Singly Linked List for Dynamic Playlist Management
66 class Node:
67     def __init__(self, data):
68         self.data = data
69         self.next = None
70 class SinglyLinkedList:
71     def __init__(self):
72         self.head = None
73     def append(self, data):
74         new_node = Node(data)
75         if not self.head:
76             self.head = new_node
77             return
78         last_node = self.head
79         while last_node.next:
80             last_node = last_node.next
81         last_node.next = new_node
82     def display(self):
83         current_node = self.head
84         while current_node:
85             print(current_node.data)
86             current_node = current_node.next
```



```
85         print(current_node.data)
86         current_node = current_node.next
87 # Example usage of Singly Linked List for playlist management
88 if __name__ == "__main__":
89     playlist = SinglyLinkedList()
90     # Adding songs to the playlist
91     playlist.append("Song 1: Imagine - John Lennon")
92     playlist.append("Song 2: Bohemian Rhapsody - Queen")
93     playlist.append("Song 3: Stairway to Heaven - Led Zeppelin")
94     print("Current Playlist:")
95     playlist.display()
```

OUTPUT:

```
PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai eax.py"
Current Playlist:
Song 1: Imagine - John Lennon
Song 2: Bohemian Rhapsody - Queen
Song 3: Stairway to Heaven - Led Zeppelin
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> █
```

Task 4: Binary Search Tree for Fast Record Lookup

Scenario:

You are building a student record system where quick searching by roll number is required.

Prompt: Binary Search Tree for Fast Record Lookup

```

96 #Binary Search Tree for Fast Record Lookup
97 class TreeNode:
98     def __init__(self, key):
99         self.left = None
100         self.right = None
101         self.val = key
102 class BinarySearchTree:
103     def __init__(self):
104         self.root = None
105     def insert(self, key):
106         if self.root is None:
107             self.root = TreeNode(key)
108         else:
109             self._insert_recursively(self.root, key)
110     def _insert_recursively(self, node, key):
111         if key < node.val:
112             if node.left is None:
113                 node.left = TreeNode(key)
114             else:
115                 self._insert_recursively(node.left, key)
116         else:
117             if node.right is None:
118                 node.right = TreeNode(key)
119             else:
120                 self._insert_recursively(node.right, key)

```

```

102 class BinarySearchTree:
110     def _insert_recursively(self, node, key):
116         else:
117             if node.right is None:
118                 node.right = TreeNode(key)
119             else:
120                 self._insert_recursively(node.right, key)
121     def inorder_traversal(self):
122         return self._inorder_recursively(self.root)
123     def _inorder_recursively(self, node):
124         res = []
125         if node:
126             res = self._inorder_recursively(node.left)
127             res.append(node.val)
128             res = res + self._inorder_recursively(node.right)
129         return res
130 # Example usage of Binary Search Tree for record lookup
131 if __name__ == "__main__":
132     bst = BinarySearchTree()
133     # Inserting records (keys)
134     bst.insert(50)
135     bst.insert(30)
136     bst.insert(70)
137     bst.insert(20)
138     bst.insert(40)
139     bst.insert(60)
140     bst.insert(80)
141     print("Inorder Traversal of BST (Sorted Keys):")
142     print(bst.inorder_traversal())

```

OUTPUT:

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell + - [] ... | []

```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai/ea.py"
Inorder Traversal of BST (Sorted Keys):
[20, 30, 40, 50, 60, 70, 80]
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>

```

Task 5: Graph Traversal for Social Network Connections

Scenario:

You are modeling a social network, where users are connected to friends, and you want to explore connections.

Prompt: Graph Traversal for Social Network Connections

```

141         print('Inorder Traversal of BST (Sorted Keys):')
142         print(bst.inorder_traversal())'''
143 #Graph Traversal for Social Network Connections
144 class Graph:
145     def __init__(self):
146         self.graph = {}
147     def add_edge(self, u, v):
148         if u not in self.graph:
149             self.graph[u] = []
150         if v not in self.graph:
151             self.graph[v] = []
152         self.graph[u].append(v)
153         self.graph[v].append(u) # Assuming undirected graph
154     def bfs(self, start):
155         visited = set()
156         queue = [start]
157         while queue:
158             vertex = queue.pop(0)
159             if vertex not in visited:
160                 print(vertex)
161                 visited.add(vertex)
162                 queue.extend([neighbor for neighbor in self.graph[vertex] if neighbor not in visited])
163     def dfs(self, start, visited=None):
164         if visited is None:
165             visited = set()
166         if start not in visited:
167             print(start)
168             visited.add(start)
169             for neighbor in self.graph[start]:
170                 self.dfs(neighbor, visited)
171
172 class Graph:
173     def bfs(self, start):
174         while queue:
175             vertex = queue.pop(0)
176             if vertex not in visited:
177                 print(vertex)
178                 visited.add(vertex)
179                 queue.extend([neighbor for neighbor in self.graph[vertex] if neighbor not in visited])
180     def dfs(self, start, visited=None):
181         if visited is None:
182             visited = set()
183         if start not in visited:
184             print(start)
185             visited.add(start)
186             for neighbor in self.graph[start]:
187                 self.dfs(neighbor, visited)
188
189 # Example usage of Graph for social network connections
190 if __name__ == "__main__":
191     social_graph = Graph()
192     # Adding connections (edges) between users (vertices)
193     social_graph.add_edge("yashwanth", "Bob")
194     social_graph.add_edge("yashwanth", "Charlie")
195     social_graph.add_edge("Bob", "David")
196     social_graph.add_edge("Charlie", "Eve")
197     print("Breadth-First Search (BFS) starting from yashwanth:")
198     social_graph.bfs("yashwanth")
199     print("\nDepth-First Search (DFS) starting from yashwanth:")
200     social_graph.dfs("yashwanth")
201

```

OUTPUT:

```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai/exam
.py"
Breadth-First Search (BFS) starting from yashwanth:
yashwanth
Bob
Charlie
David
Eve

Depth-First Search (DFS) starting from yashwanth:
yashwanth
Bob
David
Charlie
Eve
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>

```

